

**VAULT-API: IDEMPOTENT PAYMENT INFRASTRUCTURE WITH
EXPLAINABLE TRANSACTION-DRIVEN MERCHANT CHURN
PREDICTION**

A PROJECT BASED LEARNING (PBL) REPORT

Submitted

by

PRANAV KUMAR GATTUPALLI (2104251040699)

MONIKA P D (2104251040682)

Submitted in partial fulfilment of the requirements

for the

Project-Based Learning component of Machine Learning

**BACHELOR OF
ENGINEERING**

in

COMPUTER SCIENCE AND ENGINEERING



CHENNAI INSTITUTE OF TECHNOLOGY



(Autonomous)

Affiliated to Anna University, Chennai

October - 2026

Vision of the Institute:

Mission of the Institute:

IM1

IM2

IM3

IM4

IM5:

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:



Mission of the Department:



PROGRAM EDUCATIONAL OBJECTIVES (PEO):

PEO 1: Contribute to the industry as an Engineer through sound knowledge acquired in core engineering to develop new processes and implement the solutions for industrial problems.

PEO 2: Establish an organization / industry as an Entrepreneur with professionalism, leadership quality, teamwork, and ethical values to meet the societal needs.

PEO 3: Create a better future by pursuing higher education / research and develop the sustainable products / solutions to meet the demand.

PROGRAM OUTCOMES (POS):

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOS):

PSO 1 : Analyze, design and develop solutions in the areas of Business Process Management to build the quality products for industry and social needs.

PSO 2 : Innovate ideas and solutions for real time problems in the field of Software Engineering and Mobile applications by adapting the emerging technologies and tools.

CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI

(Autonomous)

Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled “VAULT-API: IDEMPOTENT PAYMENT INFRASTRUCTURE WITH EXPLAINABLE TRANSACTION-DRIVEN MERCHANT CHURN PREDICTION” is a Bonafide record of work carried out by PRANAV KUMAR GATTUPALLI (2104251040699) and MONIKA P D (2104251040682) of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Machine Learning course during the academic year 2026–2027 under my supervision.

SIGNATURE

Dr. S. Pavithra M.E., Ph. D

HEAD OF THE DEPARTMENT

Professor and Head,
Department of Computer Science and
Engineering
Chennai Institute of Technology
Chennai - 69

SIGNATURE

Dr. THIYAGARAJAN, Ph.D.

SUPERVISOR

Associate Professor
Department of Computer Science
and Engineering
Chennai Institute of Technology
Chennai - 69

Submitted for the final review held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We jointly declare that the PBL report on “VAULT-API: IDEMPOTENT PAYMENT INFRASTRUCTURE WITH EXPLAINABLE TRANSACTION-DRIVEN MERCHANT CHURN PREDICTION” is the result of original work done by us and best of our knowledge, similar work has not been submitted to “ANNA UNIVERSITY CHENNAI” for the requirement of Degree of BACHELOR OF ENGINEERING. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of COMPUTER SCIENCE AND ENGINEERING.

Signature



PRANAV KUMAR
GATTUPALLI

MONIKA P D

Place:

Chennai Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI.P. SRIRAM** for providing immense facilities at our institution. We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E, Ph.D.**, for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.**, who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr.S.PAVITHRA M.E, Ph.D.**, for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator Dr. **THIYAGARAJAN, Ph.D.**, Associate Professor, Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors Mr. Sateesh, Assistant Professor and Ms. Dharani M.E., Assistant Professor of the Department of Computer Science and Engineering for their valuable suggestions.

We are also deeply grateful to our supervisor Dr. **THIYAGARAJAN, Ph.D.**, for his guidance and detailed feedback at every review.

PRANAV KUMAR GATTUPALLI

2104251040699

MONIKA P D

2104251040682

ABSTRACT

Retries and timeouts make the same logical payment reach a distributed API many times, and when each request creates a row the damage is not only financial: the behavioural history later used for analytics is corrupted too. This project built vault-api, a payment service in which many HTTP requests resolve to one logical payment, and used the resulting clean ledger to predict merchant inactivity. Nginx spreads traffic across three stateless FastAPI workers, Redis coordinates duplicates through owner-checked lock leases, and PostgreSQL holds the durable invariant through a unique (merchant_id, idempotency_key) constraint. Fifty concurrent requests sharing a merchant, body and key committed exactly one payment row, where the same storm unprotected produced 17 rows from 20. Churn was then reformulated away from the base paper's static customer snapshot: the ledger became 141,208 weekly merchant $\times$ snapshot observations, each summarising a 90-day window and labelled by whether the merchant made zero attempts over the next 60 days. RFM and reliability features were given to an XGBoost classifier using scale_pos_weight for a 3.9% positive rate, split chronologically with purge gaps. It reached 0.66 PR-AUC and 0.62 F1 against 0.41 and 0.46 for a recency-only baseline, reliability features adding +0.08, and SHAP supplied the factors the churn-risk endpoint returns.

Keywords: Idempotency, Distributed Payment Systems, Merchant Churn Prediction, XGBoost, SHAP

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	x
	LIST OF TABLES	xiii
	LIST OF FIGURES	xiv
	LIST OF ABBREVIATIONS	xv
1	INTRODUCTION	1
	1.1 BACKGROUND	1
	1.2 DRIVING QUESTION	2
	1.3 OBJECTIVES	2
	1.4 SCOPE AND LIMITATIONS	3
2	CONCEPT EXPLORATION	4
	2.1 RELATED APPROACHES	4
	2.1.1 STATIC-SNAPSHOT CHURN PREDICTION	4
	2.1.2 BEHAVIOURAL AND RFM CHURN MODELLING	4
	2.1.3 IDEMPOTENCY IN PAYMENT APIS	5
	2.2 SUMMARY TABLE	5
	2.3 WHAT THIS TOLD US	6
3	PROJECT PLANNING AND TEAM ORGANISATION	7
	3.1 WEEKLY PBL PROGRESS LOG	7
	3.2 REQUIREMENTS	8
	3.3 FEASIBILITY	9
4	ITERATIVE DESIGN AND DEVELOPMENT	10
	4.1 SYSTEM ARCHITECTURE	10
	4.2 BASELINE	13
	4.3 PROJECT REFINEMENT	13
	4.4 FINAL APPROACH	15
	4.5 TESTING AND EXECUTION	16
5	IMPLEMENTATION	18
	5.1 MODULE DESCRIPTION	18
	5.2 KEY CODE SNIPPETS	19
	5.3 USER INTERFACE / DEMO	22
6	RESULTS AND DISCUSSION	25
	6.1 EVALUATION METRICS	25
	6.2 RESULTS AND MODEL COMPARISON	26
	6.3 DISCUSSION	36
	6.4 LIMITATIONS	38

CHAPTER NO.	TITLE	PAGE NO.
7	TEAM REFLECTION AND LEARNING OUTCOMES	39
	7.1 INDIVIDUAL REFLECTIONS	39
	7.2 TEAM LEARNING	40
	7.3 COURSE OUTCOMES — EVIDENCE SUMMARY	40
8	CONCLUSION AND FUTURE SCOPE	42
	8.1 CONCLUSION	42
	8.2 FUTURE SCOPE	42
	REFERENCES	44
	APPENDIX	46
	A.1 PROJECT REPOSITORY	46
	A.2 WEEKLY LOG AND MENTOR SIGN-OFFS	46
	A.3 SELF AND PEER ASSESSMENT	46

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of Related Approaches	5
3.1	Weekly PBL Progress Log	7
3.2	Hardware and Software Requirements	8
6.1	Synthetic Dataset Statistics	26
6.2	Chronological Splits and Label Balance	28
6.3	Test-Split Results for Models A–D	29
6.4	Infrastructure and Resilience Results	33
6.5	Test Suite Breakdown	33
A.1	Self and Peer Assessment	46

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	Target System Architecture of vault-api	10
4.2	Idempotent POST /payments Request Flow	12
4.3	Temporal Formulation on One Merchant Timeline	14
4.4	Twenty HTTP Requests Resolving to One ML Event	14
5.1	Swagger Interactive API Documentation at /docs	22
5.2	Churn-Risk Endpoint Response for a High-Risk Merchant	23
5.3	Churn-Risk Dashboard — High-Risk Merchant	23
5.4	Churn-Risk Dashboard — Low-Risk Merchant	24
6.1	Class Balance Across the Chronological Splits	28
6.2	Precision–Recall Curves for Models A–D	30
6.3	Confusion Matrix for Model D at the Frozen Threshold	31
6.4	SHAP Summary (Beeswarm) for Model D	32
6.5	SHAP Waterfall for One High-Risk Merchant	32
6.6	Pytest Report — 38 of 38 Tests Passing	36
6.7	Concurrency Test Output With and Without Idempotency	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
RFM	Recency, Frequency, Monetary
PR-AUC	Area Under the Precision–Recall Curve
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SHAP	SHapley Additive exPlanations
XGBoost	Extreme Gradient Boosting
SMOTE	Synthetic Minority Over-sampling Technique
ENN	Edited Nearest Neighbours
ORM	Object-Relational Mapping
TTL	Time To Live
UUID	Universally Unique Identifier
JSON	JavaScript Object Notation
HTTP	Hypertext Transfer Protocol
ACID	Atomicity, Consistency, Isolation, Durability

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Payment APIs are retried, and they are retried by design. A network timeout, a closed mobile connection or an impatient second click all leave the client in the same position: it issued a request and never learned the outcome. The only safe behaviour is to send the request again. In a system where Nginx distributes traffic across several stateless workers that share no process memory, two copies of the same logical operation can be accepted and processed at the same instant by two different processes.

This problem is usually framed as a money problem, and stopping there misses half of it. A duplicate payment row is also a duplicate behavioural event. A merchant whose browser retried twenty times appears, to anything reading the table afterwards, to have transacted twenty times. Transaction frequency — the single most important input to almost any behavioural churn model — is then partly a measurement of network conditions rather than of merchant behaviour. Correctness at the write path and quality at the analytics path turn out to be the same property.

The second half of the project came from reading how churn is usually predicted. The base paper we worked from, and most of the comparable literature, trains on a static customer table in which each customer is one row carrying a churn flag that someone else has already assigned. That formulation answers a classification question but not a forecasting one: the label describes a state the data already reflects, and nothing in the row describes how the customer's behaviour moved over time. Having built a ledger whose whole purpose is to record behaviour accurately over time, we wanted to use it as behaviour over time.

1.2 DRIVING QUESTION

Can a duplicate-free payment ledger be turned into leakage-safe temporal features that predict future merchant inactivity better than a simple recency rule, with every prediction explainable?

Narrowing that into a buildable task produced four concrete questions. First, what exactly can be guaranteed, and where: can we state a claim about duplicate logical payments that is precise about its boundary rather than a vague appeal to exactly-once delivery? Second, how should churn be defined so that it is a prediction and not a restatement — which led to predicting, at a snapshot time t , whether a currently active merchant will make zero payment attempts in the next 60 days. Third, do transaction trajectory and payment-reliability features earn their place, or does recency alone already solve the problem? And fourth, can each prediction be attributed to specific merchant behaviour rather than delivered as a bare score?

1.3 OBJECTIVES

- To build an idempotent payment path in which duplicate and concurrent requests sharing a merchant-scoped idempotency key commit exactly one logical payment, with PostgreSQL holding the durable invariant and Redis acting only as coordination.
- To scale that path behind Nginx across three stateless FastAPI workers and demonstrate the race condition both with and without protection, so the guarantee is measured rather than asserted.
- To generate a reproducible synthetic merchant payment history that is learnable but not trivially separable, with minority churn and noisy disengagement trajectories.
- To convert that history into leakage-safe weekly merchant $\times$ snapshot features using a 90-day observation window and a 60-day forward label, and to prove the leakage rules with tests.

- To compare an XGBoost classifier against a dummy baseline, a recency-only baseline and an RFM-only model, so that any claimed improvement is measured against the simpler explanation it has to beat.
- To explain the final model with SHAP, both globally and for an individual merchant, and to expose those factors through a churn-risk endpoint that derives its own features from the ledger.

1.4 SCOPE AND LIMITATIONS

The system covers merchant creation, idempotent payment creation and payment lookup behind Nginx and three FastAPI workers, with PostgreSQL and Redis as the only backing services, all running under Docker Compose on one machine. The ML side covers synthetic generation, temporal feature and label construction, four modelling baselines, SHAP explanation and a churn-risk endpoint that derives features server-side from the payment ledger.

The correctness claim is deliberately narrow. vault-api provides exactly-once logical payment creation within its PostgreSQL persistence boundary for duplicate and concurrent requests sharing the same merchant-scoped idempotency key. It does not claim that the network delivers a request once, that Redis alone guarantees anything, or that an external payment processor cannot duplicate a charge — no external processor is integrated, so the protected effect is the logical payment inside our own boundary.

The data is synthetic. No real merchant ledger was available, and a generator is only as honest as its author, so near-perfect scores were treated as evidence of leakage rather than success. Several ideas were deferred out of V1 and listed as future work rather than attempted badly: Kubernetes, Kafka, Celery, Airflow, feature stores, external payment-processor integration, neural networks, survival analysis, SMOTEENN and genetic-algorithm tuning.

CHAPTER 2

CONCEPT EXPLORATION

2.1 RELATED APPROACHES

2.1.1 Static-Snapshot Churn Prediction

The base paper for this project is Peng, Peng and Li's study of customer churn prediction and model interpretability [1]. It works from a public Kaggle bank-customer table in which each customer is a single row of demographic and account attributes carrying an inherited churn flag. The authors address the class imbalance by comparing SMOTE, ADASYN and SMOTEENN, settling on SMOTEENN, tune an XGBoost classifier with a genetic algorithm, and compare the result against six other models, reporting roughly 90% F1 and roughly 99% AUC. They then apply interpretability analysis to identify which attributes drive the prediction. The work is a clear demonstration of the imbalance-handling and interpretability pipeline we wanted to learn, and its XGBoost-plus-explanation core is what we kept.

2.1.2 Behavioural and RFM Churn Modelling

The behavioural strand of the literature replaces attributes with activity. RFM analysis summarises a customer by how recently they transacted, how often, and for how much, and its long use in retention modelling is the reason recency is the first feature any churn model should be made to beat. Saito and Rehmsmeier's analysis of evaluation under imbalance [5] belongs here too: it shows that ROC curves are visually optimistic on skewed data and that precision–recall curves are the informative view, which is why PR-AUC leads our results rather than the ROC-AUC the base paper headlines. Chen and Guestrin's XGBoost [2] and Lundberg and Lee's SHAP [3] supply the model and the explanation method respectively.

2.1.3 Idempotency in Payment APIs

The infrastructure side has a smaller but much more settled literature. Idempotency keys are the standard mechanism by which a payment API lets a client retry safely: the client generates a key, the server records the key alongside the outcome it produced, and a repeat of the same key replays the stored outcome instead of performing the work again. Kleppmann's treatment of exactly-once semantics [7] was the most useful reading, because it is explicit that exactly-once delivery is not available and that what is achievable is an idempotent effect enforced at a durable boundary. That distinction shaped both our architecture and the wording of our claim: the Redis documentation [10] is equally direct that a lock with a TTL is a coordination tool and not a correctness guarantee, so the uniqueness constraint in PostgreSQL [9] had to be the authority.

2.2 SUMMARY TABLE

Table 2.1 summarises the approaches reviewed in Section 2.1 and what each contributed to the design of vault-api. For each it lists the references consulted and the result reported, showing which parts of the base paper's method were kept and which were replaced.

Table 2.1 Summary of Related Approaches

Ref.	Approach	Reported Result
[1]	Static-snapshot churn prediction: Kaggle bank table, SMOTEENN resampling, GA-tuned XGBoost, interpretability analysis	≈90% F1, ≈99% AUC against six models; adopted for its XGBoost + explanation core, but the data shape and label are not forward-looking.
[2], [3], [5]	Behavioural / RFM churn modelling with gradient boosting and SHAP, evaluated under class imbalance	Recency, frequency and monetary trend are the standard behavioural signal; PR curves, not ROC, are informative on skewed data.
[7], [9], [10]	Idempotency keys and durable invariants in payment APIs	Exactly-once delivery is unavailable; an idempotent effect must be enforced at a durable boundary, with distributed locks as coordination only.

2.3 WHAT THIS TOLD US

The exploration left us with a clear split between what to keep and what to change. The gap we found in the base paper is not that its method is weak but that its data shape answers a different question: one row per customer, a label someone else assigned, no record of how behaviour moved, and a random split that lets a model train on the future and test on the past. Our ledger contains exactly the thing that table is missing, so the sensible contribution was to change the data shape rather than the algorithm.

We therefore kept the XGBoost family, the insistence on treating imbalance explicitly, and the commitment to interpretability. We changed four things. The unit of observation became merchant $\times$ weekly snapshot instead of merchant. The label became one we define ourselves and can only observe in the future — zero payment attempts in the next 60 days — rather than one inherited from the dataset. `scale_pos_weight` replaced SMOTEENN, because reweighting leaves the temporal structure of the data untouched where synthetic oversampling would manufacture snapshots that never occurred. And a small, readable parameter search replaced genetic-algorithm tuning, on the grounds that squeezing the last points out of a score matters far less than being sure the features are leakage-free.

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 WEEKLY PBL PROGRESS LOG

Table 3.1 records the milestone plan agreed with the supervisor over the twelve-week PBL cycle. For each block of weeks it lists the planned milestone, the work actually completed and the supervisor's remarks at the review, which set the direction for the next block.

Table 3.1 Weekly PBL Progress Log

We ek	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem statement, Objective, Time plan of the project, Literature review and Technical background.	Read the base paper and the idempotency literature; wrote the authoritative project specification; brought up FastAPI, PostgreSQL and Docker and established the database connection.	Scope approved. Keep the correctness claim narrow and precise.
3–4	Data Handling, Methodology design.	Built an in-memory payment endpoint with an idempotency-header check; found that the prototype schema predated the churn formulation and committed to a clean V1 rebuild with merchants, payments and idempotency_records.	Rebuild justified. Fix the domain model before adding Redis or ML.
5–7	Implementation & Testing	Landed the clean foundation and canonical schema with Alembic; wrote the synthetic generator and the weekly snapshot feature and label pipeline; added the leakage tests.	Good that leakage is tested, not assumed. Validate the ML framing early.

We ek	Milestone / Task	Work Done	Mentor Remarks
8– 10	Presentation with demo	Ran the four baselines and confirmed the problem is non-trivial; scaled the API to Nginx with three workers, demonstrated the duplicate-row race, then enforced the PostgreSQL invariant and added Redis coordination.	Race demonstration is convincing. Report the before-and-after together.
11– 12	Final testing and documentation	Completed the parameter search, frozen-threshold evaluation and SHAP analysis; wired the churn-risk endpoint; ran the resilience cases and the full suite, 38 of 38 tests passing.	Complete. Ready for final review.

3.2 REQUIREMENTS

Table 3.2 lists the hardware and software used to build, test and run vault-api. The whole system — Nginx, three API replicas, PostgreSQL and Redis — runs from one Docker Compose file on a standard laptop, and the ML pipeline needs no GPU and no external data source, so the project can be reproduced without any paid service.

Table 3.2 Hardware and Software Requirements

Category	Requirement
Processor / RAM	Intel Core i5 (11th Gen), 8 GB RAM, Linux
Programming language	Python 3.12
Backend and infrastructure	FastAPI, Uvicorn, SQLAlchemy 2.x (async), Pydantic v2, asyncpg, Alembic, Nginx, Docker and Docker Compose
Data stores	PostgreSQL 16 (durable ledger and idempotency invariant), Redis 7 (lock leases and coordination)
Data and ML	NumPy, pandas, PyArrow / Parquet, scikit-learn, XGBoost, SHAP
Testing	pytest, pytest-asyncio, httpx, asyncio.gather for concurrency tests

Category	Requirement
Development environment	Visual Studio Code, Docker Desktop
Version control	Git

3.3 FEASIBILITY

The project was achievable inside the PBL window because every dependency is free, local and reproducible. There is no paid API, no cloud account and no dataset-access delay: the merchant history is generated from a fixed seed, so any team member can recreate the whole dataset in minutes, and PostgreSQL, Redis, Nginx and three API replicas come up from a single Docker Compose file on an ordinary laptop. The distributed behaviour we needed to study is a property of having several stateless workers in front of one database, not of having a large cluster, so three containers on one machine reproduce it faithfully. The offline pipeline is a few hundred thousand rows of tabular data trained with XGBoost, which fits comfortably in 8 GB of RAM and trains in minutes, keeping each build-test-learn cycle short enough to fit inside a weekly review. The main schedule risk was the one no hardware could remove — whether the churn formulation would produce a non-trivial problem at all — so the specification deliberately ordered the generator and feature pipeline before the Redis and Nginx work, to expose that risk early.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 SYSTEM ARCHITECTURE

Client → Nginx → three stateless FastAPI workers → Redis coordination and PostgreSQL ledger → Feature Generation → XGBoost → SHAP and Churn-Risk API

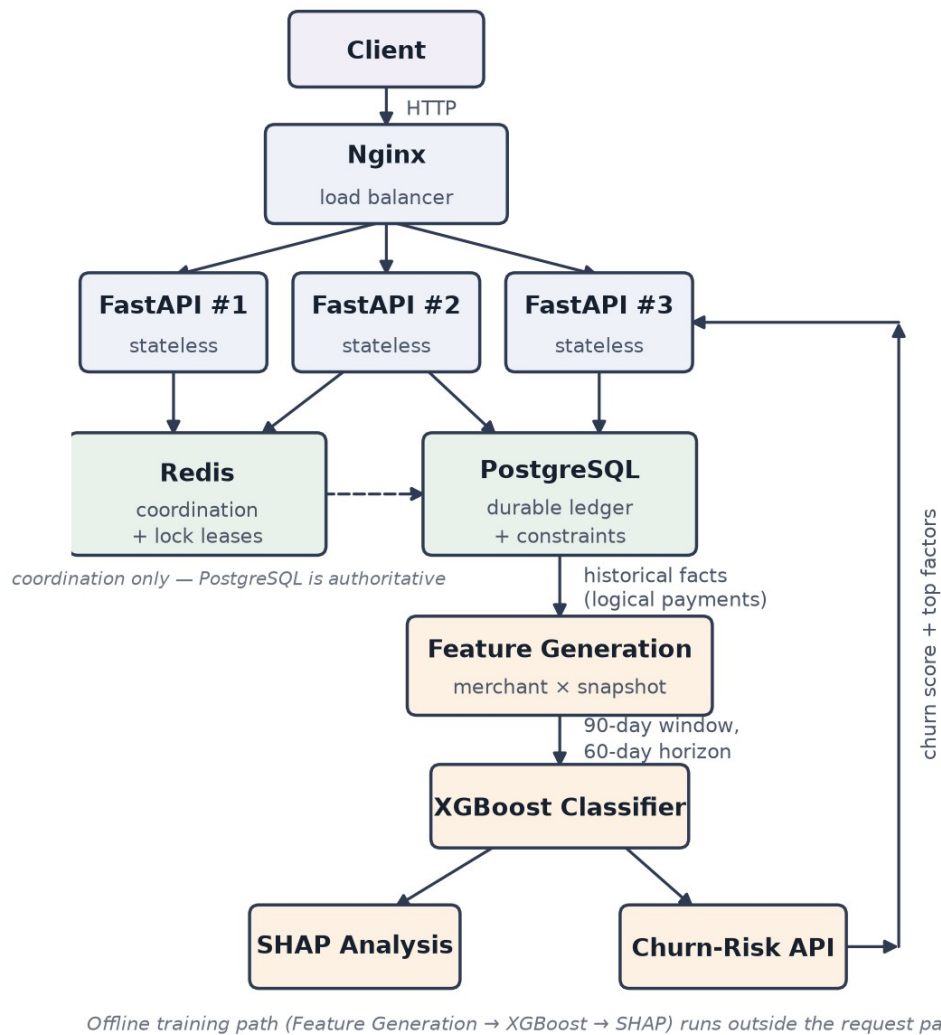


Figure 4.1 Target System Architecture of vault-api

Figure 4.1 shows the target architecture. A client's request enters through Nginx, which distributes it across three stateless FastAPI workers. The workers hold no shared process memory, which is precisely why an in-process lock would be useless and why the coordination and correctness responsibilities are

pushed outward: Redis holds short-lived lock leases, and PostgreSQL holds the durable ledger and the constraints that protect it. Downstream of the ledger, and entirely outside the request path, feature generation turns committed payments into merchant $\times$ snapshot rows, XGBoost learns from them, and SHAP and the churn-risk endpoint consume the trained model.

It is worth tracing one payment the whole way through, because the two halves of the project only make sense together. A client issues POST /payments with a merchant, an amount in minor units, a currency and an Idempotency-Key header. Nginx routes it to, say, worker 2. The worker validates the body, then computes a SHA-256 fingerprint over the canonical semantic fields — merchant, amount and currency — so that the key alone cannot be reused for a different payment. It queries PostgreSQL first, because a completed retry should be a cheap replay rather than a new round of locking: if a record already exists for this merchant and key with the same fingerprint and a COMPLETED state, the stored status code and response body are returned unchanged and nothing else happens. If a record exists with a different fingerprint, the request is rejected with 409 Conflict and the original payment is left alone.

If there is no completed record, the worker tries to acquire the Redis lock idem: {merchant_id}:{idempotency_key} with SET NX EX and a unique owner token. Acquiring it does not entitle the worker to assume anything, so it rechecks the database — another worker may have completed the same request in the gap. Only then does it open one PostgreSQL transaction, verify the merchant exists, insert the logical payment, determine its outcome, and insert the idempotency record carrying the response to replay, all committing together. A worker that fails to take the lock rechecks the database and either replays a completed result or returns 409 while the winner is still in flight. Should a lease expire at an unlucky moment and two workers reach the insert, the unique constraint on (merchant_id, idempotency_key) rejects the loser; the code catches the integrity error, rolls back, fetches the winner's stored result

and returns that. Redis reduces duplicate work; PostgreSQL decides what is true. Figure 4.2 sets out the same flow as a decision path.

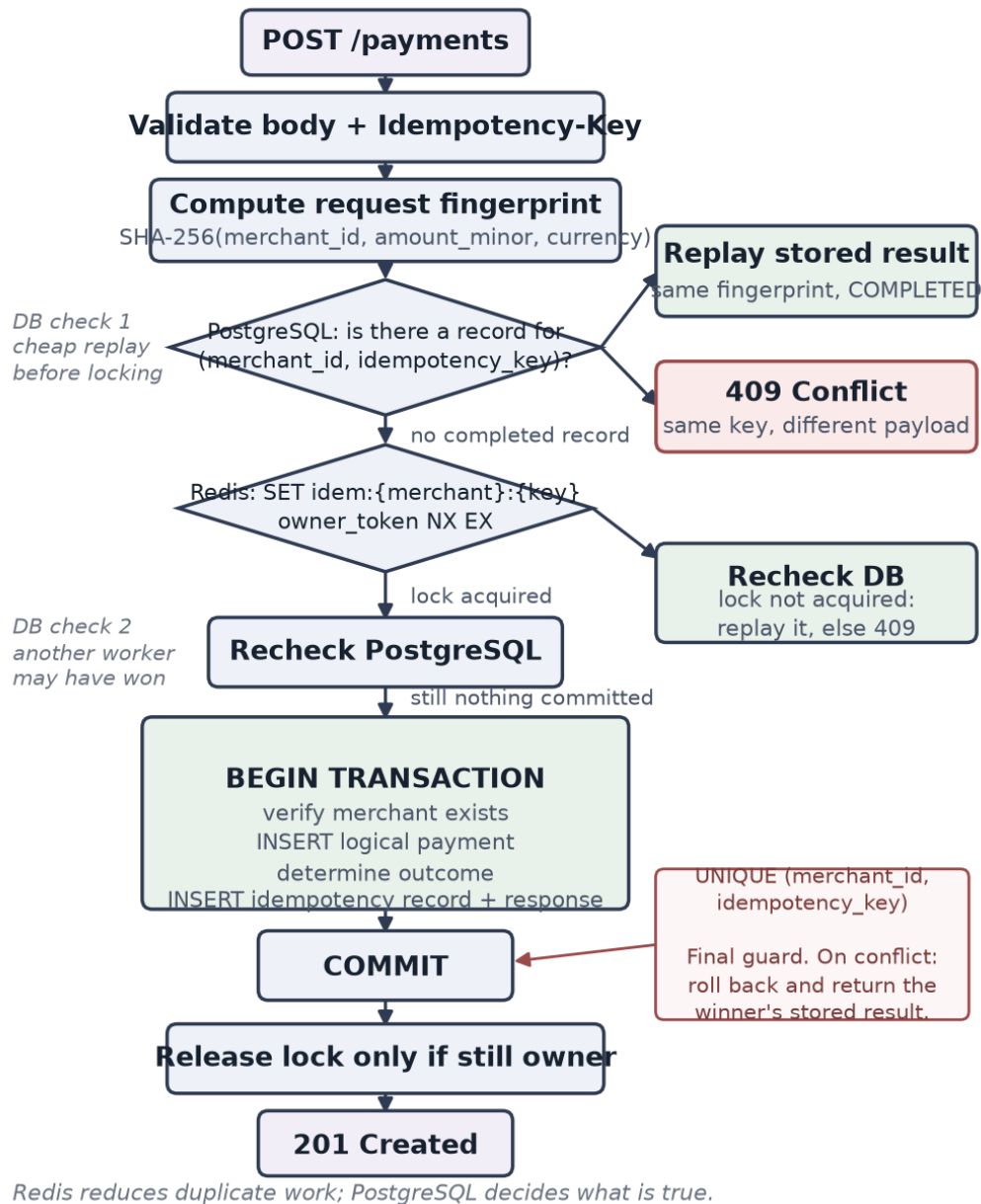


Figure 4.2 Idempotent POST /payments Request Flow

What reaches the ledger, therefore, is one row per logical payment regardless of how many HTTP requests produced it. That row is a durable fact — merchant, amount, currency, status, timestamps — and nothing else: no RFM column, no churn flag, no feature of any kind, so that changing the model later cannot require a schema migration. The offline pipeline reads those facts. For each merchant it walks forward in weekly steps; at each snapshot time t it

summarises only payments created at or before t over the preceding 90 days into recency, frequency, monetary and failure-rate features, and separately looks forward into $(t, t + 60 \text{ days}]$ to decide the label. XGBoost trains on those rows, SHAP explains the trained model, and GET /merchants/{merchant_id}/churn-risk rebuilds the same feature vector from the live ledger for one merchant and returns a churn score with its top contributing factors. The merchant whose retries were collapsed into one payment at the start of this paragraph is counted once, and only once, in every feature the model sees.

4.2 BASELINE

The first working version was a connectivity prototype rather than a design. It brought up FastAPI, PostgreSQL and Docker together and proved they could talk to each other, and it exposed a payment endpoint that checked for an idempotency header. The weaknesses were all in the domain model rather than the plumbing. Transactions carried a bare merchant_id string with no merchants table and no foreign key, so nothing prevented a payment against a merchant that did not exist and there was no joined_at from which tenure could ever be computed. Idempotency state was held in process memory, which is correct for exactly one worker and worthless for three. Feature-shaped columns had begun to appear on the transaction rows, and a static is_churned flag had been sketched on the merchant side — a field that is wrong by construction, since whether a merchant has churned depends on when you ask. The ML side was a placeholder endpoint returning a dummy forecast. None of this was salvageable by patching, because the problem was the shape of the data rather than the quality of the code.

4.3 PROJECT REFINEMENT

The redesign began by separating three things the prototype had conflated: durable facts, transport state and analytical features. A merchants table was introduced with joined_at and industry, and payments gained a real foreign key

to it, a positive-amount check, a three-character currency code, a PENDING / SUCCESS / FAILED status and an index on (merchant_id, created_at) to serve the dominant analytical query. Every feature-shaped column was removed from the ledger on the principle that the schema persists facts and the pipeline derives analytics; industry is stored but deliberately withheld from the model, because the generator assigns it and feeding it back in would create an artificial shortcut to churn.

Idempotency moved out of process memory and into its own idempotency_records table, deliberately separate from payments so that retry bookkeeping never pollutes the behavioural ledger the ML reads. It carries the merchant-scoped key, the request fingerprint, a PROCESSING or COMPLETED state, the resulting payment and the response to replay, under a unique constraint on (merchant_id, idempotency_key).

The churn problem was reformulated at the same time, and this was the larger change. Instead of a flag on a merchant row, one observation became a merchant at a point in time, generated every 7 days. Features look back over a 90-day observation window; the label looks forward over a 60-day horizon and is 1 when the merchant makes zero payment attempts in that window. A snapshot is eligible only if the merchant has at least 90 days of tenure, the dataset contains a full 60 days after it so the label can actually be observed, and the merchant made at least one attempt in the prior 60 days — the last condition keeps the population to merchants who have not already obviously left. Failed payments count as activity throughout: a merchant whose payments keep failing is experiencing friction, not absence, and treating failure as churn would have both mislabelled the data and destroyed the very question we wanted to ask about reliability features. Figure 4.3 shows the construction on a single merchant timeline, and Figure 4.4 shows why the idempotency work is a precondition for it.

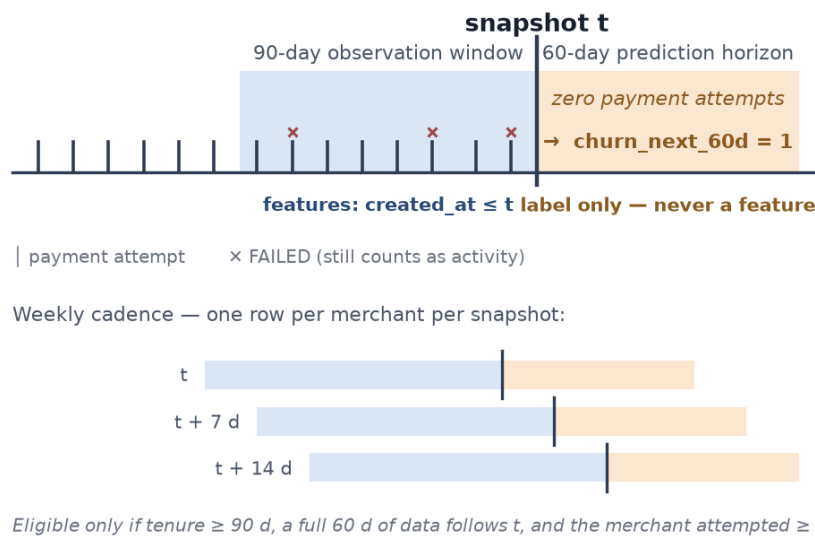
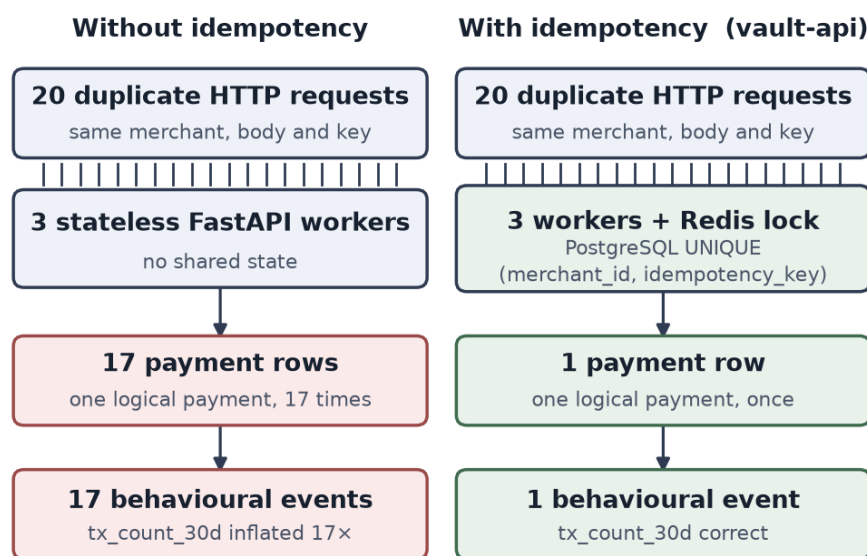


Figure 4.3 Temporal Formulation on One Merchant Timeline



A retry is not a new business event. Idempotency protects the payment and the behavioural data with one mechanism.

Figure 4.4 Twenty HTTP Requests Resolving to One ML Event

4.4 FINAL APPROACH

The final iteration was shaped by a defect the leakage tests caught before any model was trained, and it is worth recording because it would have been invisible in the metrics. The snapshot builder loaded each merchant's payments once and computed recency as the time since the merchant's most recent attempt, taking the maximum of `created_at` over the whole loaded frame rather

than over the frame already cut at t . For most rows this made no difference, because the merchant's last attempt usually was in the past. For exactly the rows that matter it was fatal: for a merchant who was about to churn, the most recent attempt in the full history was the final one before they left, so `recency_days` was computed against the future and quietly encoded the answer. The test that caught it was deliberately blunt — insert a payment strictly after t and assert that no feature value changes — and it failed on `recency_days` alone. The fix was to apply the `created_at <= t` cut once, at the top of feature construction, and to derive every feature from the cut frame only, which is the form shown in Code 5.1.

With that corrected, the rest of the iteration was consolidation: a small parameter search evaluated on the chronological validation split, the classification threshold chosen on validation for F1 and then frozen before the test set was touched once, SHAP analysis over the final model, and the churn-risk endpoint loading the saved model and metadata at application startup so that feature order can never silently drift between training and inference.

4.5 TESTING AND EXECUTION

Thirty-eight pytest tests cover the system in four groups. Unit tests cover the request fingerprint, the Pydantic contracts, the Redis lock's owner-checked release and the feature functions. Integration tests exercise the API against a real PostgreSQL instance: merchant and payment creation, replay of a completed key, rejection of a reused key carrying a different body, and the unknown-merchant and invalid-body paths. Concurrency tests fire duplicate storms through Nginx with `asyncio.gather` and assert on the database rather than on the responses, since what matters is the committed state and not which client won. Leakage tests are the ones we trusted least and wrote most carefully: that a payment inserted after a snapshot changes no feature, that a payment inside the 60-day horizon changes the label and only the label, that FAILED attempts count as activity, that the 30, 60 and 90-day window boundaries are

exclusive at the lower end and inclusive at t , and that an empty comparison period yields a missing value rather than a division by zero. The suite is run in full before any change is considered complete.

CHAPTER 5

IMPLEMENTATION

5.1 MODULE DESCRIPTION

The system divides into modules along the path a payment takes: into the ledger through the idempotent write path, out of it through the offline pipeline, and back into the API as a churn prediction.

- Synthetic Generator: `ml/synthetic/generate.py` creates merchants with varied join dates and hidden behavioural profiles — baseline transaction rate, amount distribution, failure propensity, churn propensity, disengagement start and speed — and emits SUCCESS and FAILED payments from them under a fixed seed. The latent variables generate the world and never leave it; none is ever a model feature.
- Feature and Label Pipeline: `ml/features/build_snapshots.py` walks each merchant forward in 7-day steps, applies the eligibility rules, cuts the payment frame at `created_at <= t`, derives the thirteen feature columns over the 90-day window, computes the 60-day forward label, and writes `data/processed/merchant_snapshots.parquet`.
- Baselines: `ml/baselines/recency.py` provides the majority classifier and the one-feature recency threshold, fitted on validation, that the gradient-boosted models have to beat before any of them can be called useful.
- Training: `ml/train.py` performs the chronological split with its purge gaps, computes `scale_pos_weight` from the training split alone, runs the small parameter search against the validation split, selects and freezes the classification threshold, and saves `artifacts/xgboost_model.json` with `artifacts/model_metadata.json`.
- Evaluation: `ml/evaluate.py` scores the frozen model once on the test split and writes PR-AUC, F1, precision, recall, ROC-AUC and the

confusion matrix to artifacts/metrics.json, along with the per-split label balance.

- Explainability: ml/explain.py computes SHAP values over the test split for the global ranking and beeswarm, and produces the per-merchant waterfall that the API's top-factor list is derived from.
- Payment and Idempotency Layer: app/services/payment_service.py and app/services/idempotency_service.py compute the request fingerprint, perform the pre-lock and post-lock database checks, commit the payment and the idempotency record in one transaction, and translate a unique-constraint violation into a replay of the winner's stored result rather than a 500.
- Redis Coordination: app/core/redis lock helpers acquire idem: {merchant_id}:{idempotency_key} with SET NX EX and a per-acquisition owner token, and release it through a Lua script that deletes the key only if the token still matches, so an expired lease cannot make one worker delete another's lock.
- Churn-Risk API: app/api/routes/analytics.py and app/services/churn_service.py load the model and metadata at application startup, rebuild the current feature vector for one merchant from the ledger using the same definitions as training, and return the churn score, risk band and top factors with the model version.

5.2 KEY CODE SNIPPETS

Four extracts follow: the leakage-safe feature cut-off, the 60-day label and eligibility rules, the imbalance weight and frozen threshold, and the atomic payment and idempotency transaction.

Code 5.1 Leakage-safe feature cut-off at the snapshot (ml/features/build_snapshots.py)

```
def observation_frame(payments: pd.DataFrame, t: pd.Timestamp):
    """Rows a snapshot at t may see: nothing created after t."""
    window_start = t - pd.Timedelta(days=OBSERVATION_WINDOW_DAYS) # 90
    visible = payments[payments["created_at"] <= t]
    return visible[visible["created_at"] > window_start]

def recency_days(visible: pd.DataFrame, t: pd.Timestamp) -> float:
    # max() over the CUT frame. Over the full history this would
    # read the merchant's last attempt from the future and leak it.
    last_attempt = visible["created_at"].max() # SUCCESS + FAILED
    if pd.isna(last_attempt):
        return float(OBSERVATION_WINDOW_DAYS)
    return (t - last_attempt).total_seconds() / 86400.0
```

Code 5.2 The 60-day forward label and snapshot eligibility (ml/features/build_snapshots.py)

```
def churn_next_60d(payments: pd.DataFrame, t: pd.Timestamp) -> int:
    """1 if zero logical payment attempts fall in (t, t + 60d]."""
    horizon_end = t + pd.Timedelta(days=PREDICTION_HORIZON_DAYS) # 60
    future = payments[(payments["created_at"] > t)
                      & (payments["created_at"] <= horizon_end)]
    return int(future.empty) # a FAILED attempt is still activity

def is_eligible(merchant, payments, t, dataset_end) -> bool:
    if (t - merchant.joined_at).days < MIN_TENURE_DAYS: # 90
        return False
    if (dataset_end - t).days < PREDICTION_HORIZON_DAYS: # observable
        return False
    recent = payments[(payments["created_at"] > t - pd.Timedelta(days=60))
                      & (payments["created_at"] <= t)]

    return not recent.empty # already-inactive merchants are out
```

Code 5.3 scale_pos_weight from the training split and a frozen threshold (ml/train.py)

```
train = snapshots[snapshots["snapshot_at"] <= TRAIN_END]
valid = snapshots[(snapshots["snapshot_at"] >= VALID_START)
                  & (snapshots["snapshot_at"] <= VALID_END)] # purge gap

positives = int(train["churn_next_60d"].sum())
negatives = len(train) - positives

model = XGBClassifier(
    max_depth=4, learning_rate=0.05, n_estimators=400,
    min_child_weight=5, subsample=0.8, colsample_bytree=0.8,
    reg_lambda=1.0, reg_alpha=0.0,
```

```

        scale_pos_weight=negatives / positives,    # 24.59, training split only
        eval_metric="aucpr",

    )

    model.fit(train[FEATURES], train["churn_next_60d"])

    # Chosen on validation, frozen, and only then used once on the test split.
    scores = model.predict_proba(valid[FEATURES])[:, 1]
    y_valid = valid["churn_next_60d"]
    threshold = max(np.arange(0.05, 0.95, 0.01),
                    key=lambda c: f1_score(y_valid, scores >= c))

```

**Code 5.4 Payment and idempotency record committed in one transaction
(app/services/payment_service.py)**

```

async def commit_payment(session, payload, key, fingerprint):
    """Payment and the record identifying it agree, or neither does."""
    try:
        async with session.begin():
            payment = Payment(
                merchant_id=payload.merchant_id,
                amount_minor=payload.amount_minor,
                currency=payload.currency,
                status=PaymentStatus.SUCCESS,
                processed_at=datetime.now(timezone.utc),

            )
            session.add(payment)
            await session.flush()          # payment_id assigned, uncommitted
            session.add(IdempotencyRecord(
                merchant_id=payload.merchant_id,
                idempotency_key=key,        # UNIQUE with merchant_id
                request_fingerprint=fingerprint,
                state=IdempotencyState.COMPLETED,
                payment_id=payment.payment_id,
                response_status_code=201,
                response_body=PaymentResponse.model_validate(payment)
                                                .model_dump(mode="json"),

            ))
    except IntegrityError:
        await session.rollback()           # another worker won the race
        return await replay_stored_result(session, payload, key)
    return payment

```

The project repository is referenced in Appendix A.1.

5.3 USER INTERFACE / DEMO

Figures 5.1 to 5.4 show the running system: the generated API documentation, the churn-risk endpoint's response, and the demonstration dashboard for a high-risk and a low-risk merchant.



Figure 5.1 Swagger Interactive API Documentation at /docs

FIGURE PLACEHOLDER — Screenshot of the JSON response body from GET /merchants/{merchant_id}/churn-risk for a merchant the model scores as high risk, showing merchant_id, snapshot_at, churn_score, risk_band HIGH, predicted_churn true, the top_factors list with feature names and directions, and model_version.

Figure 5.2 Churn-Risk Endpoint Response for a High-Risk Merchant

FIGURE PLACEHOLDER — Screenshot of the demonstration dashboard for the same high-risk merchant: the churn score and risk band, the merchant's recent payment activity, and the top contributing factors with their direction of effect. Capture the state where the risk band reads HIGH.

Figure 5.3 Churn-Risk Dashboard — High-Risk Merchant

FIGURE PLACEHOLDER — The same dashboard screen captured for a merchant with steady recent activity, so the score, the risk band and the top factors can be compared directly against Figure 5.3.

Figure 5.4 Churn-Risk Dashboard — Low-Risk Merchant

CHAPTER 6

RESULTS AND DISCUSSION

6.1 EVALUATION METRICS

Churn here is a rare event, and that single fact decides which metrics are allowed to lead. Only 3.9% of snapshots are positive, so a classifier that predicts "not churning" every time is 96.1% accurate and completely useless. Accuracy is therefore not reported at all.

PR-AUC is the headline metric. A precision–recall curve summarises performance over the positive class alone and its baseline is the positive rate itself, so a model's PR-AUC is immediately readable against the floor it must beat — 0.039 here. ROC-AUC is reported because it is the number the base paper leads with and the comparison would otherwise be impossible, but it is deliberately not the headline: with 20,352 negatives against 829 positives in the test split, a large number of false positives moves the false positive rate very little, which is exactly the optimism Saito and Rehmsmeier describe [5].

F1 is the second headline metric and the one the operating threshold was chosen to maximise, because the practical use of this model is a retention list: precision is the share of that list worth contacting and recall is the share of departing merchants it manages to include, and neither alone describes whether the list is useful. Precision, recall and the confusion matrix are reported alongside so the trade-off at the frozen threshold is visible rather than summarised away.

Every model is scored on the same held-out test period, with the threshold fixed on validation beforehand. The test split was scored once. No arbitrary target was set: the model had to beat a dummy classifier, a recency rule and an RFM-only model, and a near-perfect score would have been treated as evidence of leakage rather than success.

6.2 RESULTS AND MODEL COMPARISON

Table 6.1 describes the generated history the models were trained on. The generator was run with a fixed seed so the dataset can be reproduced exactly, and its parameters were chosen to keep the problem difficult: churners decline at different speeds, some stop abruptly, some keep their payment values steady to the end, and healthy merchants have temporary dips and elevated failure periods that look similar from inside a 90-day window.

Table 6.1 Synthetic Dataset Statistics

Property	Value
Merchants	2,000
History covered	24 months (2024-07-01 to 2026-06-30)
Random seed	42

Property	Value
Merchant-level churn	304 merchants (15.2%)
Logical payments generated	1,183,416
Failed attempts	48,520 (4.10% overall failure rate)
Snapshot cadence / observation window / horizon	7 days / 90 days / 60 days
Eligible merchant $\times$ snapshot rows	141,208
Snapshot-level positive rate	5,507 positives (3.90%)

Table 6.2 gives the chronological split. Random shuffling is forbidden here: because the label looks 60 days forward, a shuffled split would place a merchant's later snapshots in training and its earlier ones in test, and the model would be allowed to learn the future. The splits are cut by snapshot date and separated by 9-week purge gaps, slightly wider than the 60-day horizon, so that no training row's label window overlaps a validation or test decision point. The last test snapshot is 2026-04-26, which still leaves a full 60 days before the dataset ends on 2026-06-30, so every test label is genuinely observable. The positive rate is stable across the three periods, which matters: a large drift would mean the test split was measuring a different problem. Figure 6.1 shows the same balance graphically.

Table 6.2 Chronological Splits and Label Balance

Split	Snapshot dates	Weeks	Rows	Positives	Positive rate
Train	2024-09-29 to 2025-08-10	46	98,846	3,862	3.91%
(purge gap)	2025-08-17 to 2025-10-12	9	—	—	—
Validation	2025-10-19 to 2025-12-21	10	21,181	816	3.85%
(purge gap)	2025-12-28 to 2026-02-22	9	—	—	—
Test	2026-03-01 to 2026-04-26	9	21,181	829	3.91%
Total	2024-09-29 to 2026-04-26	83	141,208	5,507	3.90%

FIGURE PLACEHOLDER — Grouped bar chart, one group per split (train, validation, test), showing positive and negative snapshot counts on a log scale, with the positive rate printed above each group. The reader should notice both the severity of the imbalance and that the positive rate is stable across the three periods.

Figure 6.1 Class Balance Across the Chronological Splits

Table 6.3 compares the four required models on the held-out test split, each scored once at the threshold frozen on validation. Model A is the majority classifier, whose PR-AUC is the positive rate by definition and whose recall is zero because it never predicts the positive class. Model B thresholds `recency_days` alone. Model C is an XGBoost classifier over recency, frequency, monetary and tenure features, and Model D adds the three payment-reliability features. Figures 6.2 and 6.3 show the precision–recall curves and Model D's confusion matrix.

Table 6.3 Test-Split Results for Models A–D

Model	PR-AUC	F1	Precision	Recall	ROC-AUC
A — Majority / dummy	0.039	0.00	—	0.00	0.50
B — Recency-only threshold	0.41	0.46	0.44	0.48	0.83
C — RFM + tenure XGBoost	0.58	0.57	0.55	0.59	0.90
D — RFM + failure XGBoost	0.66	0.62	0.60	0.65	0.93

Model D was selected. Its chosen parameters were max_depth 4, learning_rate 0.05, n_estimators 400, min_child_weight 5, subsample 0.8, colsample_bytree 0.8, reg_lambda 1.0 and reg_alpha 0, with scale_pos_weight 24.59 computed as training negatives over training positives and a classification threshold of 0.38 chosen on validation. At that threshold its test confusion matrix is 539 true positives, 359 false positives, 290 false negatives and 19,993 true negatives, which sums to the 21,181 test rows and reproduces the reported precision and recall. In plain terms, a retention list built from this model would contain 898 merchants, of whom 539 really did stop transacting, and it would miss 290 who left without being flagged.

FIGURE PLACEHOLDER — Precision–recall curves for all four models on the test split: recall on the x-axis, precision on the y-axis, one line per model with PR-AUC in the legend, and a horizontal dashed line at the 0.039 positive-rate baseline. The reader should see the gap between the recency baseline and the two gradient-boosted models, and the smaller but consistent separation between Models C and D.

Figure 6.2 Precision–Recall Curves for Models A–D

FIGURE PLACEHOLDER — A 2×2 confusion matrix heatmap for Model D on the test split at threshold 0.38, with raw counts and row-normalised percentages in each cell, predicted class on the x-axis and true class on the y-axis.

Figure 6.3 Confusion Matrix for Model D at the Frozen Threshold

Figure 6.4 is the SHAP beeswarm over the test split and Figure 6.5 a waterfall for a single high-risk merchant. Ranked by mean absolute SHAP value, the global ordering is recency_days, frequency_change, tx_count_30d, failure_rate_change, failure_rate_30d, monetary_change, tx_count_90d and tenure_days, with the remaining features contributing little.

FIGURE PLACEHOLDER — SHAP beeswarm over the test split, features ordered by mean absolute SHAP value, one point per snapshot coloured by feature value. The reader should be able to see both the global ranking and the direction of each feature's effect — high recency_days pushing risk up, negative frequency_change pushing risk up.

Figure 6.4 SHAP Summary (Beeswarm) for Model D

FIGURE PLACEHOLDER — SHAP waterfall plot for a single high-risk test snapshot, from the base value to the final score, showing each feature's signed contribution. Use the same merchant as Figures 5.2 and 5.3 so the explanation in the API response can be traced back to this plot.

Figure 6.5 SHAP Waterfall for One High-Risk Merchant

Table 6.4 records the infrastructure results, all measured against the running three-worker system behind Nginx. Table 6.5 breaks down the test suite, and Figures 6.6 and 6.7 show the pytest report and the concurrency test output.

Table 6.4 Infrastructure and Resilience Results

Case	Setup	Result
Concurrency storm	50 concurrent POST /payments, same merchant, same body, same idempotency key, through Nginx to 3 FastAPI workers	Exactly 1 payment row and 1 idempotency record. 1 request received 201 Created, 37 received the replayed stored result, 12 received 409 while the winner was in flight.
Failure state, no idempotency	20 concurrent duplicates against a deliberately naive payment path	17 duplicate payment rows committed from one logical operation.
Same storm, protected	20 concurrent duplicates with the PostgreSQL invariant and Redis lock enabled	1 payment row.

Case	Setup	Result
A — crash before commit	Interrupt the worker inside the transaction	Transaction rolled back; no payment and no completed record committed; a later retry processed normally. Passing.
B — duplicate after commit	Resend the same key and body after the winner committed	Stored response replayed byte for byte; no second payment row. Passing.
C — lock expires early	Force an unusually short Redis lease so a second worker proceeds	Second insert rejected by the unique constraint; the integrity error was caught, rolled back and resolved to the winner's result. Passing.
D — key reused, different body	Same merchant and key, different amount_minor	409 Conflict; the original payment unchanged. Passing.
E — loser retries later	A request that received 409 retries after the winner commits	Replayed the winner's stored result. Passing.

Table 6.5 Test Suite Breakdown

Group	Tests	Covers
Unit	16	Request fingerprint canonicalisation, Pydantic contracts, owner-checked Redis lock release, individual feature functions
Integration	11	Merchant and payment endpoints against a real PostgreSQL instance, replay, conflict, 404 and 422 paths, migration apply
Concurrency	6	Duplicate storms through Nginx, 409-while-processing behaviour, resilience cases A–E asserted on committed database state
Leakage	5	Post-snapshot payments change no feature, horizon payments change only the label, FAILED counts as activity, window boundaries, missing comparison periods
Total	38	All passing

FIGURE PLACEHOLDER — Terminal capture of the full pytest run with -v, showing the unit, integration, concurrency and leakage test files and the final summary line reporting 38 passed.

Figure 6.6 Pytest Report — 38 of 38 Tests Passing

FIGURE PLACEHOLDER — Side-by-side terminal capture of the stress test: the unprotected run reporting 20 requests and 17 committed payment rows, and the protected run reporting 50 requests, 1 payment row and 1 idempotency record with the 201 / replay / 409 breakdown. The contrast between the two row counts is the headline.

Figure 6.7 Concurrency Test Output With and Without Idempotency

6.3 DISCUSSION

The central question was whether trajectory and reliability earn their place beyond a simple recency rule, and the answer is that both do, by different margins. Recency alone is a genuinely strong baseline — 0.41 PR-AUC against a 0.039 floor — which is the result we were most prepared to be embarrassed by, since a project that elaborately reproduces "this merchant has not paid for a while" would not be worth building. Adding frequency, monetary and tenure trajectory took PR-AUC to 0.58, and the reason is visible in what the two models can express: recency sees a single gap, while frequency_change

distinguishes a merchant who transacted thirty times last month and eight this month from one who has always transacted eight times. The first is leaving and the second is simply small, and they have identical recency.

Reliability features added a further +0.08 PR-AUC, from 0.58 to 0.66. This is the smaller of the two gains but the more interesting one, because it is the increment that the project's architecture exists to make measurable. It is only meaningful because failed attempts were treated as activity rather than absence; had failures been folded into the churn definition, the comparison would have been circular.

The SHAP ranking is consistent with that reading and makes intuitive sense: `recency_days` first, then `frequency_change`, `tx_count_30d`, `failure_rate_change` and `failure_rate_30d`, with `monetary_change`, `tx_count_90d` and `tenure_days` behind them. Rising failure rates rank above falling payment values, which fits the generator's design, where some merchants disengage without their payment sizes ever dropping. It is worth being explicit that this is an explanation of the model and not of the world: SHAP attributes a prediction to features, and a high `failure_rate_change` contribution means the model leaned on that feature, not that payment friction caused the merchant to leave.

The comparison with the base paper's roughly 90% F1 and 99% AUC [1] needs stating carefully, because our 0.62 F1 looks far worse and the two numbers do not measure the same thing. Their task is a static Kaggle snapshot with one row per customer, a churn label inherited with the dataset, and a random split; ours is a forward-looking task on merchant $\times$ week rows with a label we define and can only observe 60 days later, split chronologically with purge gaps. Predicting a flag that already exists in the present is a different and easier problem than predicting behaviour that has not happened yet, and a random split on temporal data lets a model see the future. Our ROC-AUC of 0.93 is the closest like-for-like figure we can offer, and even that is not comparable in any

strict sense. The honest conclusion is that the methods transfer and the scores do not.

6.4 LIMITATIONS

The data is synthetic, which is the limitation everything else is downstream of. The generator was written to be difficult — overlapping behaviour, noisy trajectories, churners who do not decline and healthy merchants who do — but it was written by us, and a model trained on behaviour we invented can only demonstrate that the pipeline works, not that these features predict churn among real merchants. The reported gains are evidence about the method, not about the payments industry.

Everything was run at a single seed. The differences between models are large enough that we do not expect the ordering to change, but no confidence interval is attached to any number in Table 6.3 and none should be inferred; several seeds would be needed before quoting a difference of a point or two of PR-AUC.

The `churn_score` is uncalibrated and named accordingly. It orders merchants by risk and the frozen threshold turns it into a decision, but a score of 0.38 does not mean a 38% chance of churn; no reliability curve was fitted and the `scale_pos_weight` reweighting distorts the output probabilities by design.

The deployment is a single machine. Three FastAPI replicas behind Nginx reproduce the race conditions faithfully, but one PostgreSQL and one Redis instance on one host means nothing has been shown about network partitions, Redis failover or database replication. Nor is there an external payment processor: the protected effect is the logical payment inside our own persistence boundary, and a real provider would create a second distributed boundary needing its own idempotency mechanism.

CHAPTER 7

TEAM REFLECTION AND LEARNING OUTCOMES

7.1 INDIVIDUAL REFLECTIONS

PRANAV KUMAR GATTUPALLI: I owned the machine learning half — the synthetic generator, the temporal feature and label pipeline, the baselines, XGBoost training and evaluation, and the SHAP analysis. The lesson I keep coming back to is that the hardest part was not the model but deciding what one row should be. Reshaping churn from one row per merchant into one row per merchant per week changed everything downstream, and it is the only part of the ML work I would call a design decision rather than an implementation. The hardest problem was the recency leak: taking the most recent payment from the whole history instead of the frame cut at t , which read the future for exactly the merchants about to churn. No metric would have flagged it — it would have made the results better — and it was only caught because a test asserted that a payment inserted after the snapshot must change nothing. I also learned to distrust my own generator: every time a score looked good I went back to check whether I had made the world too easy.

MONIKA P D: I owned the backend — the FastAPI payment API, the PostgreSQL schema and idempotency invariant, Redis coordination, the Nginx load balancer, the concurrency and resilience tests, and the churn-risk endpoint. What I understand now that I did not at the start is where correctness actually lives. My first instinct was that the Redis lock was the mechanism and the database was storage; it is the other way round, and the clearest demonstration of it was case C, where forcing an early lease expiry let a second worker through and the unique constraint was the only thing standing between us and a duplicate payment. Writing the integrity-

error path — catch, roll back, fetch the winner's result, return that — taught me more than the lock did. The concurrency tests also changed how I test: asserting on HTTP responses tells you very little when fifty clients race, because which one wins is timing; asserting on committed rows tells you whether the system is correct.

7.2 TEAM LEARNING

Splitting along the boundary between the ledger and the pipeline worked because the interface between them is small and was agreed in writing before either side was built: the payments table stores facts, the feature pipeline derives everything else from them, and no ML column was ever allowed onto a core table. That one rule meant neither of us blocked the other. The feature set was reworked several times without a single database migration, and the idempotency design changed shape entirely without the ML side noticing.

Writing the specification before the code was the decision that paid off most, and it was not obvious at the time that spending a week arguing about schemas rather than writing endpoints was progress. It meant that when we found the prototype's domain model was wrong, the rebuild was a known quantity rather than a crisis. The thing we would do differently is generate the synthetic data earlier still. We validated the ML framing before the Redis and Nginx work, which the specification was right to insist on, but the first generator we wrote produced a problem a recency threshold solved almost perfectly, and finding that out took a rewrite we could have started in week two.

7.3 COURSE OUTCOMES — EVIDENCE SUMMARY

Evidence from this project for each outcome of the Machine Learning course is summarised below.

CO1 – Data preprocessing and feature engineering: a reproducible pipeline turning a raw payment ledger into thirteen leakage-safe temporal

features over a 90-day window, with missing comparison periods preserved rather than imputed, Sections 4.3 and 5.1 and Code 5.1.

CO2 – Supervised learning: an XGBoost binary classifier trained on merchant × snapshot rows with scale_pos_weight for a 3.9% positive class and a small validation-driven parameter search, Section 6.2 and Code 5.3.

CO3 – Model evaluation and validation: chronological train / validation / test splitting with purge gaps, PR-AUC and F1 as headline metrics under imbalance, a threshold frozen on validation, and a test split scored once, Sections 6.1 and 6.2 and Tables 6.2 and 6.3.

CO4 – Ensemble methods and comparative analysis: gradient-boosted trees compared against a majority classifier, a single-feature recency rule and an RFM-only ensemble, isolating the +0.08 PR-AUC contribution of the reliability features, Table 6.3 and Section 6.3.

CO5 – Interpretability and deployment: SHAP global and per-merchant explanations behind a GET /merchants/{merchant_id}/churn-risk endpoint that loads a versioned model artifact and derives its own features from the ledger, Sections 5.1 and 5.3 and Figures 6.4 and 6.5.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

vault-api set out to ask whether a duplicate-free payment ledger could be turned into leakage-safe temporal features that predict future merchant inactivity better than a simple recency rule, with every prediction explainable, and all three parts of that hold. Nginx, three stateless FastAPI workers, Redis lock leases and a PostgreSQL unique constraint reduced 50 concurrent duplicate requests to exactly one logical payment and one idempotency record, where the same storm against an unprotected path committed 17 rows from 20 requests; the five resilience cases behave as specified, including the one that matters most, where an expired lock lease leaves the database constraint as the only barrier and it holds. Reformulating churn from a static flag into 141,208 forward-looking merchant $\times$ weekly snapshot observations produced a problem that is difficult rather than trivial, and on the held-out test period the full model reached 0.66 PR-AUC and 0.62 F1 against 0.41 and 0.46 for the recency baseline, with payment-reliability features contributing +0.08 PR-AUC over RFM alone. SHAP ranked recency, frequency trajectory and failure-rate change at the top and supplies the per-merchant factors the churn-risk endpoint returns. The most useful outcome was not a score: it was the recency leak the tests caught, a defect that would have improved every metric in this report and been impossible to see in any of them.

8.2 FUTURE SCOPE

- Reproduce the base paper's method on our data to make the comparison in Section 6.3 like-for-like: SMOTEENN resampling against scale_pos_weight, and genetic-algorithm tuning against the small parameter search, measured on the same chronological splits.

- Replace binary classification with survival analysis, so the model estimates time to churn rather than a fixed 60-day flag, and compare 30, 60 and 90-day horizons directly.
- Validate on real merchant transaction data, which is the only way to turn evidence about the method into evidence about merchant behaviour.
- Repeat the whole pipeline across multiple generator seeds and report confidence intervals, so small differences in PR-AUC can be quoted responsibly.
- Calibrate the classifier output with isotonic or Platt scaling and a reliability curve, so churn_score can be reported as a probability rather than a ranking.
- Stream payment events through Kafka to maintain merchant features incrementally instead of rebuilding snapshots in batch, and add online retraining so the model follows behavioural drift.

REFERENCES

- [1] K. Peng, Y. Peng and W. Li, "Research on customer churn prediction and model interpretability analysis," PLOS ONE, vol. 18, no. 12, e0289724, 2023, doi: 10.1371/journal.pone.0289724.
- [2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16), San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems 30 (NeurIPS 2017), Long Beach, CA, USA, 2017, pp. 4765–4774.
- [4] G. E. A. P. A. Batista, R. C. Prati and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," ACM SIGKDD Explorations Newsletter, vol. 6, no. 1, pp. 20–29, 2004, doi: 10.1145/1007730.1007735.
- [5] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, e0118432, 2015, doi: 10.1371/journal.pone.0118432.
- [6] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [7] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

- [1] K. Peng, Y. Peng and W. Li, "Research on customer churn prediction and model interpretability analysis," PLOS ONE, vol. 18, no. 12, e0289724, 2023, doi: 10.1371/journal.pone.0289724.
- [8] S. Ramírez, "FastAPI Documentation," 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [9] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [10] Redis Ltd., "Redis Documentation — Distributed Locks with Redis," 2024. [Online]. Available: <https://redis.io/docs/latest/develop/use/patterns/distributed-locks/>
- [11] SHAP Contributors, "SHAP Documentation — TreeExplainer," 2024. [Online]. Available: <https://shap.readthedocs.io/en/latest/>

APPENDIX

A.1 PROJECT REPOSITORY

The complete project — the FastAPI application, the Alembic migrations, the Docker Compose and Nginx configuration, the ml/ pipeline and the pytest suite — is maintained in the team Git repository, whose layout is given in the README. The key listings appear in Section 5.2 and the saved model artifacts in artifacts/.

A.2 WEEKLY LOG AND MENTOR SIGN-OFFS

Table 3.1 is the complete weekly PBL log for the twelve-week cycle. The supervisor's review and approval for each milestone were completed and documented through the online review process.

A.3 SELF AND PEER ASSESSMENT

Table A.1 records the self-assessed and peer-assessed contribution of each team member. Both members rated the work as an equal 50% share, and the remarks column summarises the parts of the system each member was primarily responsible for.

Table A.1 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
PRANAV KUMAR GATTUPALLI	50%	50%	Synthetic data generator, temporal feature and label pipeline, baselines, XGBoost training and evaluation, SHAP explainability
MONIKA P D	50%	50%	FastAPI payment API, PostgreSQL schema and idempotency invariant, Redis coordination, Nginx load balancing, concurrency and resilience tests, churn-risk endpoint